HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN 1

BÁO CÁO BÀI TẬP AI SERVICE 02

Môn: Kiến trúc và thiết kế phần mềm

Giảng viên:	PGS TS. Trần Đình Quế
Nhóm học phần:	N08
Nhóm bài tập lớn:	04
Họ tên sinh viên:	Đoàn Quang Minh
Mã sinh viên:	B22DCCN527
Lớp:	D22CNPM06

Hà Nội 2026

Contents

1 Giới thiệu AI Service **1**
1.1 Mục tiêu và phạm vi . . . 1
1.2 Vai trò trong kiến trúc microservices . . . 1
1.3 Công nghệ sử dụng . . . 2
1.4 Endpoints chính . . . 2
1.5 Luồng xử lý tổng quát . . . 3

2 Mô tả tập dữ liệu hành vi `data_user500.csv` **4**
2.1 Tổng quan . . . 4
2.2 Thống kê cơ bản . . . 4
2.3 Các trường dữ liệu . . . 4
2.4 Quy trình tạo dữ liệu giả (Fake Data Generation) . . . 5
2.5 Phân bố nhãn hành vi (event_type) . . . 6
2.6 Ví dụ 20 dòng dữ liệu . . . 6
2.7 Nhận xét . . . 7

3 Xây dựng và đánh giá mô hình dự đoán hành vi kế tiếp (RNN/LSTM/BiLSTM) **8**
3.1 Bài toán và ý nghĩa trong E-commerce . . . 8
3.2 Tiền xử lý dữ liệu . . . 8
3.2.1 Sắp xếp và tạo chuỗi . . . 8
3.2.2 Mã hoá nhãn và đặc trưng . . . 9
3.3 Kiến trúc mô hình . . . 9
3.3.1 RNN . . . 9
3.3.2 LSTM . . . 9
3.3.3 BiLSTM . . . 9
3.4 Huấn luyện và đánh giá . . . 9
3.4.1 Thiết lập huấn luyện . . . 9
3.4.2 Metric . . . 10
3.5 Kết quả thực nghiệm . . . 10
3.5.1 Biểu đồ learning curve . . . 10
3.5.2 So sánh accuracy và macro-F1 . . . 11
3.5.3 Diễn giải kết quả . . . 12
3.5.4 Confusion matrix . . . 12
3.6 Nhận xét . . . 15
3.7 Code triển khai: định nghĩa mô hình, huấn luyện và đánh giá . . . 15
3.7.1 (5) Định nghĩa mô hình RNN/LSTM/BiLSTM . . . 15
3.7.2 (6) Huấn luyện và lưu best . . . 17
3.7.3 (7) Đánh giá trên tập test và confusion matrix . . . 18

4 Triển khai GraphRAG trong AI Service: VectorDB (FAISS) + KB_Graph (Neo4j) + LLM (Gemini) 20
4.1 Mục tiêu 20
4.2 Kiến trúc tổng thể 20
4.3 Knowledge Base và Vector Store 21
4.3.1 Cấu trúc knowledge base 21
4.3.2 Chunking và embedding 21
4.4 Neo4j: thiết kế đồ thị và Cypher query 22
4.4.1 Import dữ liệu vào Neo4j 22
4.4.2 Truy vấn context theo user_id và product_type 24
4.5 Quy trình xử lý câu hỏi 24
4.6 Mã nguồn cốt lõi của pipeline 25
4.6.1 Hàm `build_kb_index`: xây FAISS index cho knowledge base 25
4.6.2 Hàm `fetch_graph_context`: truy vấn Neo4j lấy top products 26
4.6.3 Hàm `fetch_catalog_products`: gọi Product Service lấy catalog . . . 27
4.6.4 Hàm `answer`: điều phối RAG + graph + catalog 28
4.7 Tích hợp context vào prompt và ràng buộc đề xuất 30
4.8 Ví dụ minh hoạ (bằng chứng từ hệ thống) 30
4.9 Kết luận chương 31

5 Tích hợp AI Service vào hệ thống E-commerce 32
5.1 Vai trò trong API Gateway 32
5.2 Tính năng gợi ý sản phẩm theo lịch sử tương tác (Recommendations) 32
5.3 Tính năng chatbot AI (RAG) 33
5.3.1 Giao diện và trải nghiệm người dùng 33
5.3.2 Các tình huống hỗ trợ mua sắm 34
5.4 Đánh giá tích hợp 34
5.5 Kết luận 35

Chapter 1

Giới thiệu AI Service

1.1 Mục tiêu và phạm vi

AI Service là tập hợp các chức năng AI trong hệ thống e-commerce (Django + Docker Compose) nhằm nâng cao trải nghiệm mua sắm và khả năng chuyển đổi. Trong bài tập này, AI Service tập trung vào **2 nhiệm vụ chính**:

- **Gợi ý sản phẩm dựa trên hành vi người dùng** (Recommendations): hiển thị danh sách sản phẩm ưu tiên theo lịch sử tương tác (VIEWED, CARTED, WISHLISTED, PURCHASED) tại trang `/recommendations/` của API Gateway.
- **Chatbot AI (RAG)**: trả lời câu hỏi và tư vấn sản phẩm bằng tiếng Việt, bám sát dữ liệu hệ thống, kết hợp truy xuất tri thức (knowledge base) và cá nhân hoá theo hành vi.

Các thành phần AI được thiết kế theo hướng **MVP** với các mục tiêu:

- Trả lời câu hỏi bằng tiếng Việt, ngắn gọn và bám sát dữ liệu hệ thống.
- Tư vấn và gợi ý dựa trên danh sách sản phẩm (catalog) từ `product_service`.
- Tăng độ chính xác câu trả lời bằng **RAG** (retrieval-augmented generation) trên kho tri thức nội bộ (knowledge base dạng markdown).
- Cá nhân hoá câu trả lời dựa trên hồ sơ hành vi (fake profiles) và/hoặc context từ đồ thị Neo4j (KB_Graph).
- Vẫn hoạt động được khi thiếu API key LLM: fallback rule-based.

Phạm vi của báo cáo tập trung vào: (1) mô tả dữ liệu hành vi người dùng giả lập, (2) mô hình dự đoán hành vi kế tiếp (RNN/LSTM/BiLSTM) phục vụ phân tích/đo lường, (3) triển khai GraphRAG (VectorDB + Neo4j + Gemini) cho chatbot, và (4) tích hợp AI Service vào các trang `/recommendations/` và chat trong API Gateway.

1.2 Vai trò trong kiến trúc microservices

Hệ thống được tách thành nhiều service độc lập, mỗi service có Dockerfile/requirements riêng và database độc lập (không có khoá ngoại liên-service, chỉ trao đổi qua REST).

AI Service (đặc biệt là `ai_chat_service`) đóng vai trò **trợ lý mua sắm** và **công cụ gợi ý**:

- Nhận câu hỏi từ frontend (API Gateway) kèm context (tab hiện tại: LAPTOP/CLOTHES, trang đang đứng, v.v.).
- Truy xuất tri thức liên quan trong `knowledge_base/` bằng vector search.
- Lấy danh sách sản phẩm phù hợp từ `product_service`.
- (Tuỳ chọn) Lấy context hành vi từ Neo4j để cá nhân hoá.
- Tổng hợp và sinh câu trả lời bằng LLM (Gemini-2.5-flash) hoặc fallback rule-based.

Đối với trang `/recommendations/`, API Gateway gọi AI service để lấy kết quả gợi ý. Kết quả gồm (1) graph context từ Neo4j và (2) danh sách sản phẩm recommended từ catalog; sau đó gateway render danh sách cho người dùng.

1.3 Công nghệ sử dụng

Bảng 1.1 tóm tắt các thành phần công nghệ quan trọng của AI Service (trọng tâm là service `ai_chat_service`).

Table 1.1: Công nghệ trong AI Service (`ai_chat_service`)

Thành phần	**Mô tả**
Framework	Django (REST endpoint cho chatbot)
Vector store	FAISS local (lưu tại `vector_store/`)
Knowledge base	Markdown trong `knowledge_base/` (faq/guides/policies/products)
Graph DB	Neo4j (KB_Graph và hành vi người dùng)
LLM	Gemini-2.5-flash (nếu có `GEMINI_API_KEY`); nếu không thì fallback
Inter-service calls	REST qua thư viện `requests` để gọi catalog từ `product_service`
Embeddings	MVP dùng TF-IDF + TruncatedSVD (nhẹ, không kéo PyTorch/CUDA) để tạo vector dense

1.4 Endpoints chính

Table 1.2: Các endpoint của `ai_chat_service`

Method	**Endpoint**	**Chức năng**
GET	`/api/v1/chat/health/`	Health-check
POST	`/api/v1/chat/message/`	Nhận `user_id`, `message`, `context` và trả về câu trả lời + sản phẩm gợi ý

1.5 Luồng xử lý tổng quát

Luồng xử lý của chatbot được thực hiện theo các bước:

1. Nhận request từ frontend: `user_id`, `message`, `context`.
2. Xác định `product_type` theo tab hiện tại (mặc định LAPTOP).
3. Truy vấn Neo4j để lấy `graph_context` (top products theo VIEWED, CARTED, WISHLISTED, PURCHASED).
4. Gọi `product_service` để lấy danh sách sản phẩm phù hợp với truy vấn (search term, budget, preferred brands).
5. Truy xuất knowledge base bằng FAISS (top-k đoạn liên quan) để bổ sung tri thức.
6. Sinh câu trả lời bằng Gemini (nếu có key), nếu không dùng fallback dựa trên dữ liệu truy xuất được.

Các chương tiếp theo sẽ đi sâu vào dữ liệu, mô hình dự đoán hành vi, và triển khai GraphRAG + tích hợp hệ thống.

Chapter 2

Mô tả tập dữ liệu hành vi `data_user500.csv`

2.1 Tổng quan

Tập dữ liệu `data_user500.csv` mô phỏng lịch sử tương tác của người dùng trong một website thương mại điện tử. Mỗi dòng trong file CSV tương ứng với một sự kiện (event) phát sinh trong quá trình người dùng tìm kiếm sản phẩm, xem chi tiết, thêm vào wishlist/cart, và thực hiện checkout/mua hàng.

Trong phạm vi bài báo cáo này, dataset được sử dụng cho hai mục tiêu chính:

- Phân tích hành vi và mô tả luồng tương tác (funnel) của người dùng trong e-commerce.
- Xây dựng mô hình dự đoán *hành vi kế tiếp* của người dùng dựa trên chuỗi hành vi trước đó (Chương 3).

2.2 Thống kê cơ bản

Các thống kê dưới đây được trích xuất trực tiếp từ file CSV trong project:

- Tổng số dòng (tương tác): **19,816**.
- Số người dùng (`user_id`) duy nhất: **500**.
- Số phiên (`session_id`) duy nhất: **994**.
- Số sản phẩm (`product_id`) khác nhau (không rỗng): **15**.

2.3 Các trường dữ liệu

Bảng 2.1 liệt kê các trường trong file CSV và ý nghĩa của chúng.

Table 2.1: Các trường trong `data_user500.csv`

Trường	**Ý nghĩa**
`event_id`	Mã sự kiện (định danh duy nhất cho mỗi event).
`user_id`	Mã người dùng (500 users dạng `US_001...US_500`).
`session_id`	Mã phiên truy cập; một user có thể có nhiều session.
`event_type`	Loại hành vi: `search`, `view`, `filter`, `add_to_wishlist`, `add_to_cart`, `remove_from_cart`, `checkout_start`, `purchase`.
`product_id`	ID sản phẩm liên quan (có thể rỗng với event không gắn sản phẩm).
`query_text`	Nội dung truy vấn (search/filter); rỗng với event chỉ thao tác sản phẩm.
`duration_ms`	Thời lượng thao tác (mili-giây) đối với một số event (ví dụ: xem chi tiết).
`created_at`	Thời điểm xảy ra event (ISO datetime).
`page`	Trang phát sinh sự kiện (`products_list`, `product_detail`, `cart`, `checkout`, ...).
`product_type`	Loại sản phẩm theo tab/ngữ cảnh (`LAPTOP` hoặc `CLOTHES`); trong dataset hiện tại chủ yếu là `LAPTOP`.

2.4 Quy trình tạo dữ liệu giả (Fake Data Generation)

Dữ liệu được sinh tự động bằng script:

```
neo4j/neo4j_import/generate_fake_interactions.py
```

Trong báo cáo, script này được gọi tắt là `generate_data.py`.
Ý tưởng chính:

- Tạo **hồ sơ người dùng** (UserProfile) để sinh ra chuỗi hành vi đa dạng và nhất quán theo từng người dùng.
- Sinh chuỗi hành vi theo **state machine** (ràng buộc hợp lý theo funnel):
  - `checkout_start` chỉ xảy ra khi cart có item.
  - `purchase` chỉ xảy ra khi đã `checkout_start` và cart không rỗng.
  - `remove_from_cart` chỉ xảy ra khi cart không rỗng.
  - `add_to_cart` thường sau `view` (xem chi tiết sản phẩm).
- Mỗi session có mục tiêu (*buy/browse/abandon/support*) ảnh hưởng xác suất chuyển trạng thái, tạo ra dữ liệu vừa đa dạng vừa thực tế.

2.5 Phân bố nhãn hành vi (event_type)

Phân bố số lượng theo từng hành vi được minh hoạ trong Hình 2.1. Số liệu trong dataset như sau:

Table 2.2: Phân bố event_type trong dataset

event_type	count	tỉ lệ (%)
search	6154	31.05
view	6127	30.92
add_to_wishlist	4972	25.09
filter	994	5.02
add_to_cart	930	4.69
checkout_start	240	1.21
purchase	207	1.04
remove_from_cart	192	0.97

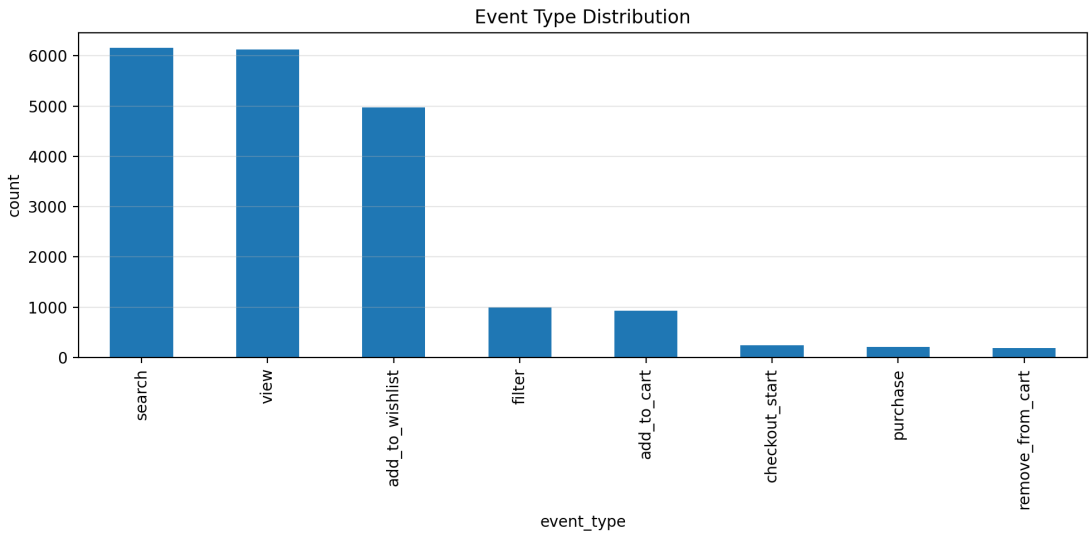

Figure 2.1: Biểu đồ phân bố hành vi (event_type) trong `data_user500.csv`

2.6 Ví dụ 20 dòng dữ liệu

Để minh hoạ trực quan cấu trúc của dataset, Hình 2.2 chụp 20 dòng dữ liệu mẫu từ `data_user500.csv`.

```csv
event_id,user_id,session_id,event_type,product_id,query_text,duration_ms,created_at,page,product_type
EVT_000001,US_001,SSN_US_001_01,remove_from_cart,11,,3456,2026-04-09T22:59:00Z,cart,LAPTOP
EVT_000002,US_001,SSN_US_001_01,search,,laptop pin tốt cho sinh viên,,2026-04-09T23:02:45Z,products_list,LAPTOP
EVT_000003,US_001,SSN_US_001_01,paginate,,,931,2026-04-09T23:03:53Z,products_list,LAPTOP
EVT_000004,US_001,SSN_US_001_01,purchase,6,,10572,2026-04-09T23:07:29Z,checkout_success,LAPTOP
EVT_000005,US_001,SSN_US_001_01,add_to_cart,2,,3228,2026-04-09T23:08:47Z,cart,LAPTOP
EVT_000006,US_001,SSN_US_001_01,checkout_start,,,7155,2026-04-09T23:11:22Z,checkout,LAPTOP
EVT_000007,US_001,SSN_US_001_01,chat,11,Trọng lượng khoảng bao nhiêu kg?,,2026-04-09T23:12:26Z,chat_widget,LAPTOP
EVT_000008,US_001,SSN_US_001_01,sort,,sort: newest,,2026-04-09T23:14:17Z,products_list,LAPTOP
EVT_000009,US_001,SSN_US_001_01,filter,,filter: price<=30000000,,2026-04-09T23:16:24Z,products_list,LAPTOP
EVT_000010,US_001,SSN_US_001_01,view,3,,34162,2026-04-09T23:17:41Z,product_detail,LAPTOP
EVT_000011,US_002,SSN_US_002_01,paginate,,,1249,2026-04-18T08:47:00Z,products_list,LAPTOP
EVT_000012,US_002,SSN_US_002_02,checkout_start,,,4621,2026-04-18T08:47:49Z,checkout,LAPTOP
EVT_000013,US_002,SSN_US_002_03,search,,laptop core i5 15-20tr,,2026-04-18T08:51:21Z,products_list,LAPTOP
EVT_000014,US_002,SSN_US_002_01,sort,,sort: price_asc,,2026-04-18T08:54:03Z,products_list,LAPTOP
EVT_000015,US_002,SSN_US_002_02,add_to_cart,7,,6381,2026-04-18T08:55:51Z,cart,LAPTOP
EVT_000016,US_002,SSN_US_002_03,view,9,,34476,2026-04-18T08:58:00Z,product_detail,LAPTOP
EVT_000017,US_002,SSN_US_002_01,filter,,filter: purpose=mỏng nhẹ,,2026-04-18T09:00:31Z,products_list,LAPTOP
EVT_000018,US_002,SSN_US_002_02,purchase,12,,5876,2026-04-18T09:03:35Z,checkout_success,LAPTOP
EVT_000019,US_002,SSN_US_002_03,remove_from_cart,15,,5198,2026-04-18T09:06:39Z,cart,LAPTOP
EVT_000020,US_002,SSN_US_002_01,chat,5,SSD 512GB có đủ dùng không hay nên 1TB?,,2026-04-18T09:10:01Z,chat_widget,LAPTOP
EVT_000021,US_003,SSN_US_003_01,sort,,sort: newest,,2026-04-04T09:27:00Z,products_list,LAPTOP
```

Figure 2.2: 20 dòng dữ liệu mẫu trong `data_user500.csv`

2.7 Nhận xét

Dataset có tính chất **mất cân bằng nhãn**: các hành vi browsing (`search/view/wishlist`) chiếm đa số, trong khi các hành vi ở cuối funnel (`checkout_start/purchase`) ít hơn đáng kể. Đây là đặc trưng phổ biến trong e-commerce thực tế và ảnh hưởng trực tiếp đến cách đánh giá mô hình (cần thêm macro-F1, confusion matrix).

Chapter 3

Xây dựng và đánh giá mô hình dự đoán hành vi kế tiếp (RNN/LSTM/BiLSTM)

3.1 Bài toán và ý nghĩa trong E-commerce

Trong hệ thống thương mại điện tử, chuỗi tương tác của người dùng thường tuân theo dạng **funnel**: `search` → `view` → `add_to_cart` → `checkout_start` → `purchase`.

Từ góc độ AI, việc dự đoán **hành vi kế tiếp** có nhiều ứng dụng:

- Dự đoán ý định mua và kích hoạt khuyến nghị phù hợp (ví dụ: nhắc ưu đãi khi user sắp checkout).
- Tối ưu UI/UX theo trạng thái hành vi (gợi ý filter/so sánh sản phẩm khi user browsing).
- Cung cấp feature cho hệ thống recommendation và chatbot (cá nhân hoá theo nhịp hành vi).

Bài toán được định nghĩa như sau:

- **Input**: một chuỗi gồm L hành vi gần nhất (sequence length), kèm theo các thuộc tính ngữ cảnh như trang (page), `product_type`, `product_id`, ...
- **Output**: nhãn `event_type` của hành vi tiếp theo.

3.2 Tiền xử lý dữ liệu

3.2.1 Sắp xếp và tạo chuỗi

Các sự kiện được sắp xếp theo `user_id`, `session_id` và `created_at` để phản ánh đúng trật tự thời gian. Sau đó, mỗi user được chuyển thành các cặp (sequence, next_label):

- Sequence gồm $L = 8$ bước trước đó.
- Nhãn mục tiêu là bước kế tiếp ngay sau sequence.

Việc tách train/test được thực hiện theo user (split-by-user) nhằm tránh **data leakage** giữa các chuỗi của cùng một người dùng.

3.2.2 Mã hoá nhãn và đặc trưng

- Nhãn hành vi `event_type` được mã hoá bằng LabelEncoder thành `action_id`.
- Các đặc trưng rời rạc (categorical) được mô tả trong báo cáo gồm: `page`, `product_type`, `product_id`. Các feature này được biến đổi thành ID để đưa vào embedding.
- Đặc trưng số (numeric) tiêu biểu: `duration_ms`. Với dữ liệu thực tế, các feature time-gap và cart-size có thể hữu ích; tuy nhiên trong báo cáo này ta giữ mô tả đơn giản.

3.3 Kiến trúc mô hình

Ba kiến trúc được thử nghiệm gồm RNN, LSTM và BiLSTM. Tất cả đều dùng ý tưởng chung:

- Embedding cho các feature categorical theo từng timestep.
- Ghép (concat) embedding + numeric vector để tạo tensor đầu vào dạng (N, L, d).
- Một backbone sequence model (RNN/LSTM/BiLSTM) để học phụ thuộc theo thời gian.
- Dense + Softmax để dự đoán xác suất trên các nhãn hành vi.

3.3.1 RNN

RNN cơ bản có dạng:

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b).$$

RNN có ưu điểm đơn giản, nhanh; nhược điểm là khó nắm bắt phụ thuộc dài.

3.3.2 LSTM

LSTM bổ sung cơ chế gate để hạn chế vanishing gradient:

$$\begin{aligned}
f_t &= \sigma(W_f[h_{t-1}, x_t] + b_f), \\
i_t &= \sigma(W_i[h_{t-1}, x_t] + b_i), \\
\tilde{c}_t &= \tanh(W_c[h_{t-1}, x_t] + b_c), \\
c_t &= f_t \odot c_{t-1} + i_t \odot \tilde{c}_t.
\end{aligned}$$

3.3.3 BiLSTM

BiLSTM chạy LSTM theo hai chiều (forward/backward) rồi ghép lại, giúp mô hình tận dụng ngữ cảnh tốt hơn trong một cửa sổ quan sát cố định L.

3.4 Huấn luyện và đánh giá

3.4.1 Thiết lập huấn luyện

Các bước huấn luyện được triển khai trong notebook `behavior_sequence_models.ipynb`. Quy trình gồm:

1. Tạo tập sequence (X) và label (y).
2. Xây mô hình Keras (RNN/LSTM/BiLSTM) với early stopping.
3. Huấn luyện và lưu model tốt nhất (`model_best`).
4. Đánh giá bằng accuracy, macro-F1 và confusion matrix.

3.4.2 Metric

Với dữ liệu mất cân bằng, **macro-F1** quan trọng vì đánh giá đều cho mọi lớp (không bị lớp lớn chi phối). Trong báo cáo, ta sử dụng:

- Accuracy: tỉ lệ dự đoán đúng tổng thể.
- Macro-F1: trung bình F1 của từng lớp.
- Confusion matrix: xem nhầm lẫn giữa các hành vi.

3.5 Kết quả thực nghiệm

3.5.1 Biểu đồ learning curve

Hình 3.1 và Hình 3.2 so sánh loss/accuracy của 3 mô hình theo quá trình huấn luyện.

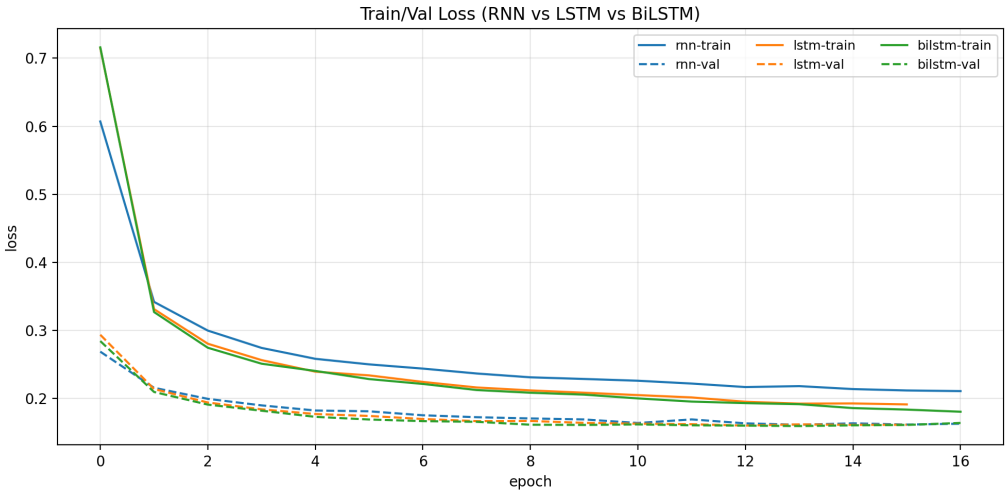

Figure 3.1: So sánh loss của RNN/LSTM/BiLSTM

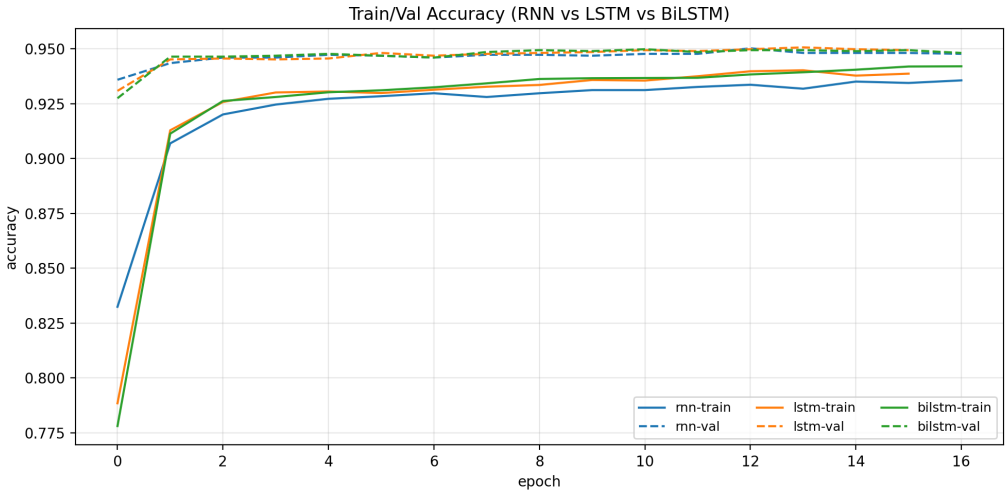

Figure 3.2: So sánh accuracy của RNN/LSTM/BiLSTM

3.5.2 So sánh accuracy và macro-F1

Hình 3.3 và Bảng 3.1 tổng hợp các chỉ số đánh giá trên tập test. Số liệu lấy từ `assets/metrics_summary.json`.

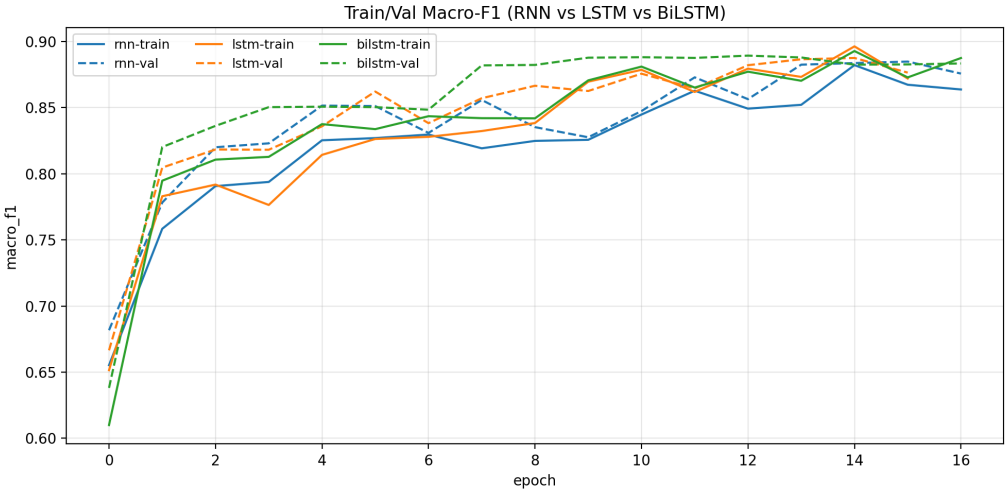

Figure 3.3: So sánh macro-F1 của RNN/LSTM/BiLSTM

Table 3.1: Kết quả đánh giá trên tập test

Mô hình	Test Accuracy	Test Macro-F1
RNN	0.9396	0.8404
LSTM	0.9379	0.8655
BiLSTM	**0.9404**	**0.8793**

Kết quả cho thấy BiLSTM đạt macro-F1 cao nhất (0.8793), do đó được chọn làm **model_best** cho bài toán dự đoán hành vi kế tiếp.

3.5.3 Diễn giải kết quả

Phần này tập trung trả lời 3 câu hỏi: (1) mô hình nào thực sự tốt hơn trong bối cảnh dữ liệu mất cân bằng, (2) mô hình học được gì từ chuỗi hành vi, và (3) lỗi chủ yếu đến từ đâu.

(1) Accuracy cao không đồng nghĩa dự đoán tốt mọi lớp. Dataset có phân bố lệch, trong đó các hành vi browsing như `search/view/add_to_wishlist` chiếm tỉ lệ lớn (Bảng 2.2).

Vì vậy, macro-F1 được dùng để đánh giá công bằng hơn trên các lớp hiếm như `add_to_cart/checkout_start/purchase`.

(2) LSTM/BiLSTM cải thiện nhờ nắm bắt phụ thuộc theo chuỗi. So với RNN, LSTM giảm hiện tượng quên thông tin ở các bước trước và ổn định hơn khi sequence dài. BiLSTM trong cửa sổ quan sát cố định $L = 8$ có thể học tốt hơn các mẫu chuyển trạng thái cục bộ, ví dụ: sau `view` thường đi kèm `add_to_wishlist` hoặc `add_to_cart`; sau `add_to_cart` mới có khả năng xảy ra `checkout_start`.

(3) Lỗi tập trung ở các lớp gần nhau trong funnel. Quan sát confusion matrix (Hình 3.4–3.6) cho thấy nhầm lẫn thường xảy ra giữa:

- `add_to_cart` và `remove_from_cart` (cùng nhóm thao tác cart).
- `add_to_cart` và `checkout_start` (gần nhau trong funnel).
- `checkout_start` và `purchase` (lớp hiếm, dễ bị ảnh hưởng bởi thiếu dữ liệu).

Kết luận chính: BiLSTM có macro-F1 tốt nhất, nghĩa là mô hình cân bằng hơn trên các lớp hiếm. Trong bối cảnh e-commerce, điều này quan trọng nếu mục tiêu là phát hiện đúng các hành vi cuối funnel (giá trị kinh doanh cao).

3.5.4 Confusion matrix

Các confusion matrix tương ứng của từng mô hình được minh hoạ trong Hình 3.4–3.6.

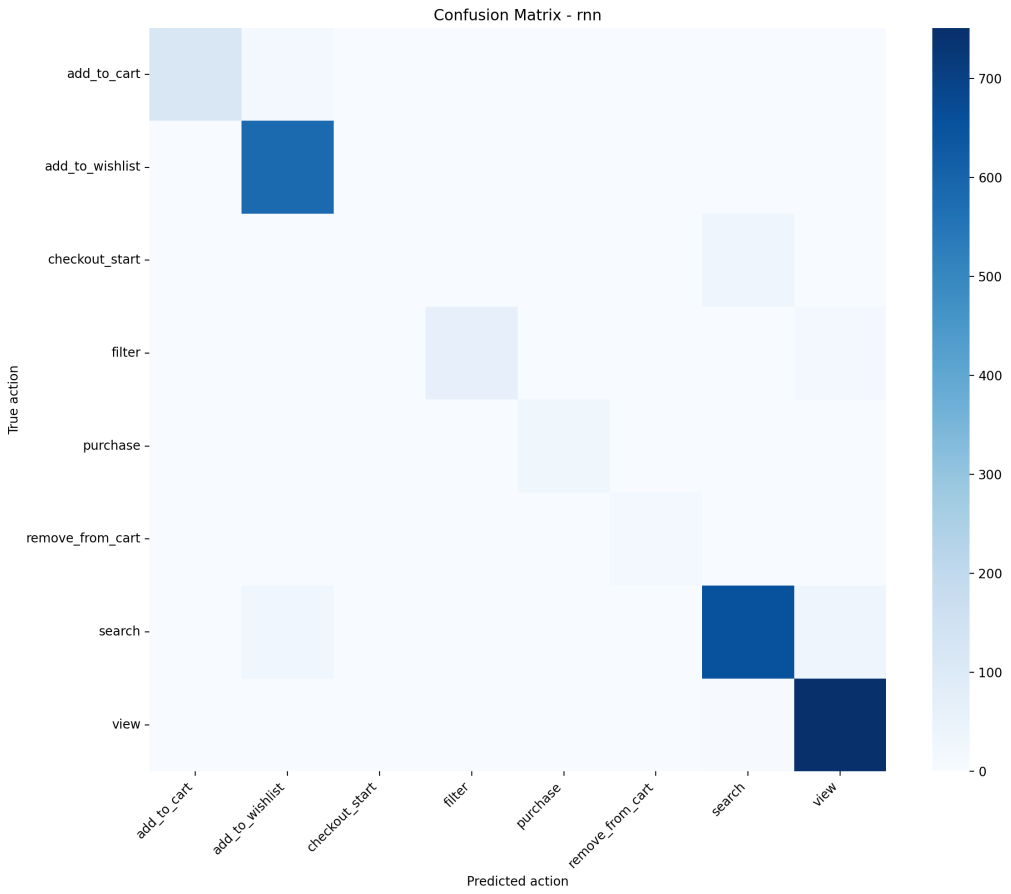

Figure 3.4: Confusion matrix của RNN

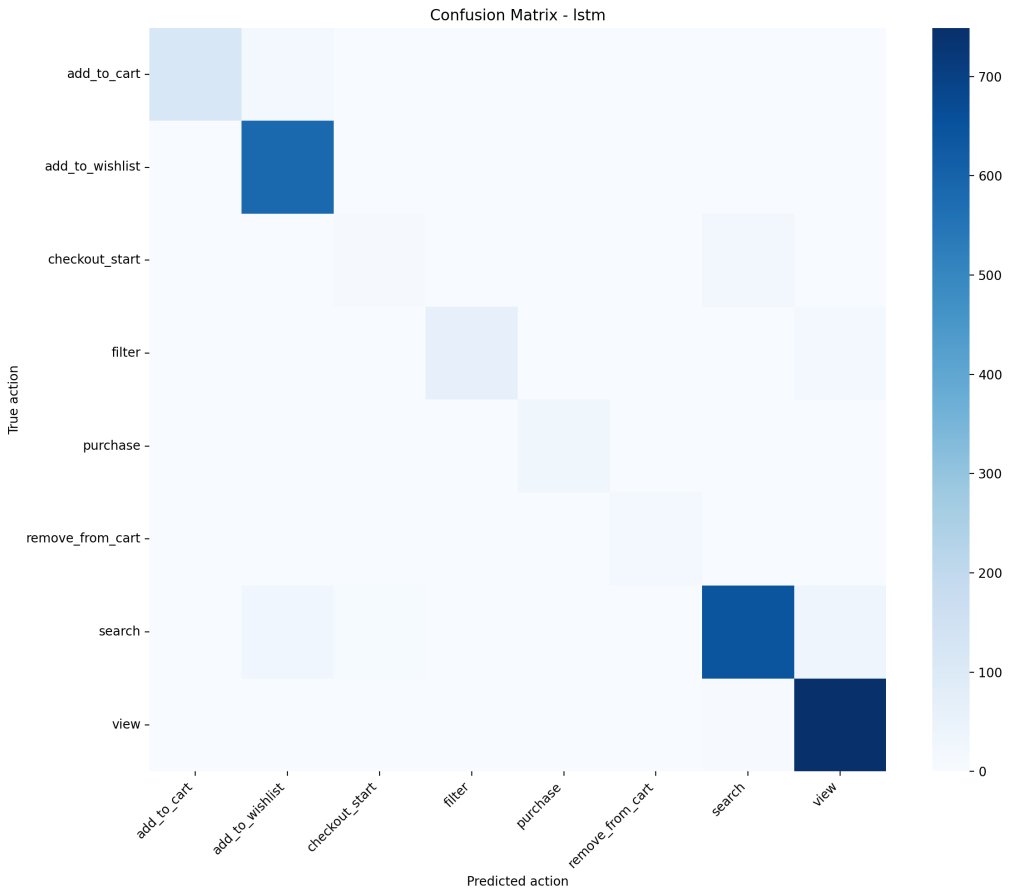

Figure 3.5: Confusion matrix của LSTM

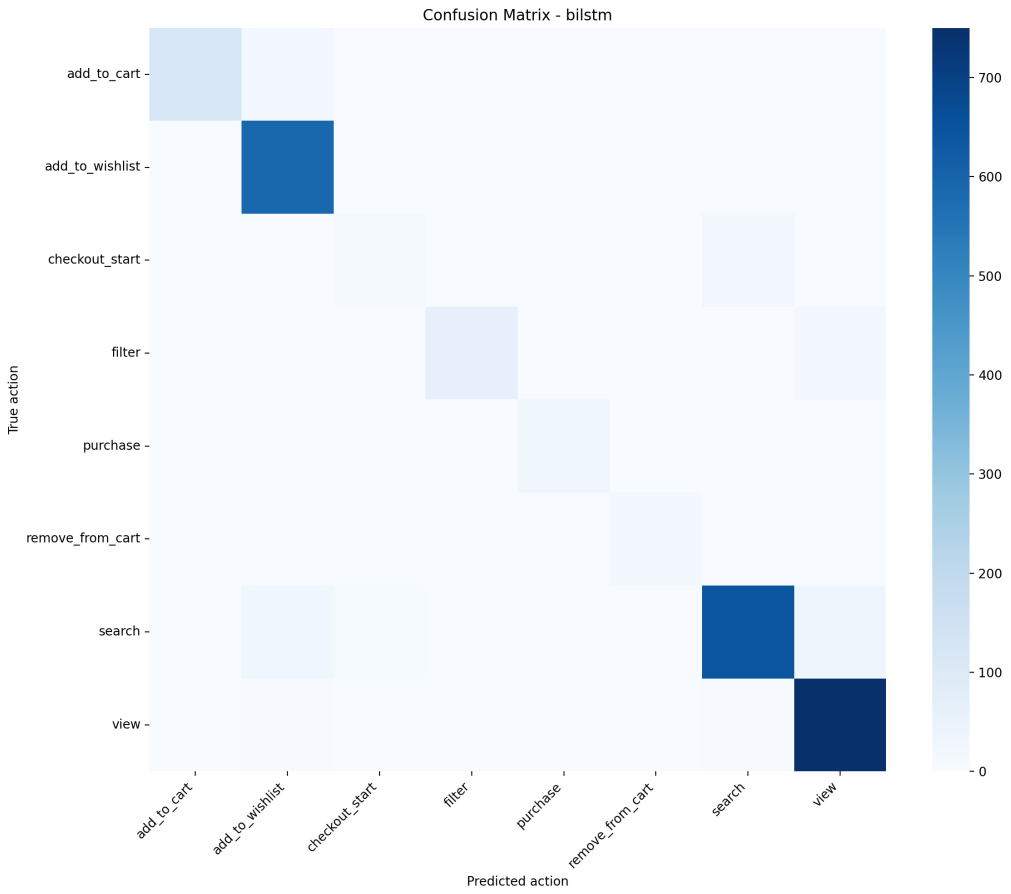

Figure 3.6: Confusion matrix của BiLSTM

3.6 Nhận xét

- Các lớp `checkout_start/purchase` ít dữ liệu nên dễ bị nhầm với các hành vi gần kề (ví dụ: `add_to_cart`).
- LSTM/BiLSTM cải thiện macro-F1 so với RNN, phản ánh khả năng học phụ thuộc chuỗi tốt hơn.
- Với dữ liệu thực tế, có thể nâng cao bằng class-weighting, focal loss, hoặc sampling theo session.

3.7 Code triển khai: định nghĩa mô hình, huấn luyện và đánh giá

Phần code tập trung vào 3 phần cốt lõi: (i) định nghĩa 3 mô hình, (ii) huấn luyện + lưu best, (iii) đánh giá test + confusion matrix.

3.7.1 (5) Định nghĩa mô hình RNN/LSTM/BiLSTM

```python
import tensorflow as tf
from tensorflow.keras import layers, Model


def build_sequence_model(
    model_type: str,
    num_actions: int,
    seq_len: int = 8,
    embed_dim: int = 32,
    rnn_units: int = 96,
    dropout: float = 0.2,
) -> Model:
    """Build one of: 'RNN', 'LSTM', 'BiLSTM'.

    Notes:
    - Đây là phiên ảbn rút ọgn để minh ạho trong báo cáo.
    - Đầu vào minh ạho: ỗchui action_id + page_id + product_id +
       duration_ms.
    """

    action_in = layers.Input(shape=(seq_len,), name="action_id")
    page_in = layers.Input(shape=(seq_len,), name="page_id")
    product_in = layers.Input(shape=(seq_len,), name="product_id")
    duration_in = layers.Input(shape=(seq_len, 1),
       name="duration_ms")

    action_emb = layers.Embedding(num_actions + 1, embed_dim,
       mask_zero=False)(action_in)
    page_emb = layers.Embedding(64, embed_dim,
       mask_zero=False)(page_in)
    product_emb = layers.Embedding(100_000, embed_dim,
       mask_zero=False)(product_in)

    x = layers.Concatenate(axis=-1)([action_emb, page_emb,
       product_emb, duration_in])
    x = layers.Dropout(dropout)(x)

    if model_type.upper() == "RNN":
        x = layers.SimpleRNN(rnn_units)(x)
    elif model_type.upper() == "LSTM":
        x = layers.LSTM(rnn_units)(x)
    elif model_type.upper() == "BILSTM":
        x = layers.Bidirectional(layers.LSTM(rnn_units))(x)
    else:
        raise ValueError(f"Unsupported model_type={model_type}")

    x = layers.Dense(128, activation="relu")(x)
    x = layers.Dropout(dropout)(x)
    out = layers.Dense(num_actions, activation="softmax",
       name="next_action")(x)
```

```
    model = Model(inputs=[action_in, page_in, product_in,
        duration_in], outputs=out)
    model.compile(
        optimizer=tf.keras.optimizers.Adam(learning_rate=1e-3),
        loss="sparse_categorical_crossentropy",
        metrics=["accuracy"],
    )
    return model
```

3.7.2 (6) Huấn luyện và lưu best

```
import numpy as np
import tensorflow as tf
from sklearn.metrics import f1_score


class MacroF1Callback(tf.keras.callbacks.Callback):
    """Compute macro-F1 on validation set after each epoch."""

    def __init__(self, val_inputs, val_labels):
        super().__init__()
        self.val_inputs = val_inputs
        self.val_labels = val_labels

    def on_epoch_end(self, epoch, logs=None):
        probs = self.model.predict(self.val_inputs, verbose=0)
        preds = np.argmax(probs, axis=1)
        macro_f1 = f1_score(self.val_labels, preds, average="macro")
        logs = logs or {}
        logs["val_macro_f1"] = macro_f1
        print(f"\nval_macro_f1: {macro_f1:.4f}")


def train_and_select_best(model, train_inputs, train_labels,
    val_inputs, val_labels):
    callbacks = [
        MacroF1Callback(val_inputs, val_labels),
        tf.keras.callbacks.EarlyStopping(
            monitor="val_loss", patience=3, restore_best_weights=True
        ),
        tf.keras.callbacks.ModelCheckpoint(
            filepath="model_best.keras",
            monitor="val_loss",
            save_best_only=True,
        ),
    ]

    history = model.fit(
        train_inputs,
        train_labels,
```

```
        validation_data=(val_inputs, val_labels),
        epochs=20,
        batch_size=256,
        callbacks=callbacks,
        verbose=1,
    )
    return history
```

3.7.3 (7) Đánh giá trên tập test và confusion matrix

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.metrics import accuracy_score, f1_score,
   confusion_matrix


def evaluate_model(model, test_inputs, test_labels, class_names):
    probs = model.predict(test_inputs, verbose=0)
    preds = np.argmax(probs, axis=1)

    acc = accuracy_score(test_labels, preds)
    macro_f1 = f1_score(test_labels, preds, average="macro")
    print(f"test_accuracy: {acc:.4f}")
    print(f"test_macro_f1: {macro_f1:.4f}")

    cm = confusion_matrix(test_labels, preds)

    fig, ax = plt.subplots(figsize=(8, 6))
    im = ax.imshow(cm, interpolation="nearest", cmap="Blues")
    ax.figure.colorbar(im, ax=ax)
    ax.set(
        xticks=np.arange(len(class_names)),
        yticks=np.arange(len(class_names)),
        xticklabels=class_names,
        yticklabels=class_names,
        ylabel="True label",
        xlabel="Predicted label",
        title="Confusion Matrix",
    )
    plt.setp(ax.get_xticklabels(), rotation=45, ha="right",
       rotation_mode="anchor")

    thresh = cm.max() / 2.0
    for i in range(cm.shape[0]):
        for j in range(cm.shape[1]):
            ax.text(
                j,
                i,
                format(cm[i, j], "d"),
                ha="center",
```

```
                va="center",
                color="white" if cm[i, j] > thresh else "black",
                fontsize=7,
            )

    fig.tight_layout()
    fig.savefig("confusion_matrix.png", dpi=200)
    return acc, macro_f1, cm
```

Chapter 4

Triển khai GraphRAG trong AI Service: VectorDB (FAISS) + KB_Graph (Neo4j) + LLM (Gemini)

4.1 Mục tiêu

Trong chatbot e-commerce, câu trả lời cần **đúng theo dữ liệu hệ thống** và có thể **giải thích được nguồn**. Vì vậy AI Service (`ai_chat_service`) triển khai kiến trúc **RAG** kết hợp **đồ thị hành vi (Neo4j)** nhằm:

- Truy xuất tri thức nội bộ (chính sách, FAQ, hướng dẫn chọn sản phẩm) từ knowledge base dạng markdown.
- Cá nhân hoá dựa trên lịch sử tương tác của user (`view/cart/wishlist/purchase`) đã import vào Neo4j.
- Ràng buộc câu trả lời: **chỉ được đề xuất sản phẩm có trong catalog** (lấy từ `product_service`).

4.2 Kiến trúc tổng thể

Kiến trúc GraphRAG trong `ai_chat_service` gồm 3 nguồn context chính (Hình 4.1):

- **VectorDB (FAISS)**: lưu embedding của các đoạn knowledge base (chunk từ file markdown).
- **KB_Graph (Neo4j)**: lưu đồ thị hành vi user và các quan hệ đến sản phẩm.
- **LLM (Gemini-2.5-flash)**: sinh câu trả lời dựa trên prompt đã được nạp context.

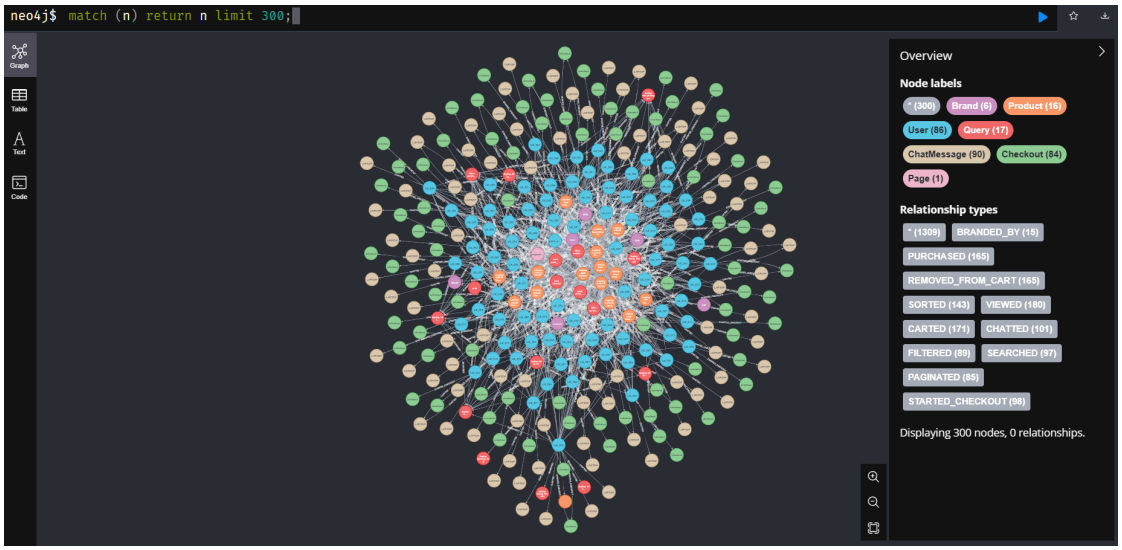

Figure 4.1: Ảnh minh hoạ đồ thị Neo4j (KB_Graph + hành vi user)

4.3 Knowledge Base và Vector Store

4.3.1 Cấu trúc knowledge base

Knowledge base được lưu trong thư mục `knowledge_base/` và chia theo nhóm nội dung:

- `faq/`: câu hỏi thường gặp (kỹ thuật, bảo hành, ...).
- `guides/`: hướng dẫn chọn laptop theo nhu cầu.
- `policies/`: chính sách đổi trả, giao hàng.
- `products/`: bài viết tham khảo theo nhóm sản phẩm (dùng để minh hoạ tư vấn).

Vector store FAISS được lưu tại `vector_store/` gồm:

- `kb.index`: chỉ mục FAISS.
- `kb_meta.json`: metadata cho từng chunk.
- `tfidf_svd.joblib`: embedder (TF-IDF + TruncatedSVD).

4.3.2 Chunking và embedding

Trong `chat/services.py`, mỗi file markdown được chia thành các đoạn (chunk) có độ dài xấp xỉ 700 ký tự với overlap 100 ký tự. Mỗi chunk được embedding bằng pipeline nhẹ:

- TF-IDF (word n-gram 1–2).
- Giảm chiều bằng TruncatedSVD (mặc định target 256).
- Chuẩn hoá vector (L2) để dùng inner-product similarity trong FAISS.

Thiết kế này giúp container gọn nhẹ (không cần PyTorch/CUDA), phù hợp mục tiêu MVP.

4.4 Neo4j: thiết kế đồ thị và Cypher query

4.4.1 Import dữ liệu vào Neo4j

Dữ liệu được import từ file CSV `data_user500.csv`. File Cypher dùng để import là `neo4j/cypher/import_interactions_500u.cypher`. Các quan hệ quan trọng:

- (User)-[:VIEWED|:CARTED|:WISHLISTED|:PURCHASED]->(Product)
- (User)-[:SEARCHED|:FILTERED]->(Query)

Mỗi quan hệ có trọng số `w` và thời điểm `last_ts` để ưu tiên hành vi gần đây / có sức nặng cao (ví dụ purchase được w cao).

Listing 4.1: Script import Neo4j (trích file `neo4j/cypher/import_interactions_500u.cypher`)

```
// Import interactions for 500 users (10 behaviors) generated by
    neo4j_import/generate_fake_interactions.py
// Neo4j reads this file as: file:///data_user500.csv
// Expected rows: approximately users * events_per_user (may vary slightly)

LOAD CSV WITH HEADERS FROM 'file:///data_user500.csv' AS row
WITH row
CALL {
  WITH row
  WITH row, datetime(row.created_at) AS ts
  MERGE (u:User {id: row.user_id})
  WITH row, ts, u,
       CASE WHEN row.product_id IS NULL OR row.product_id = '' THEN null ELSE
           toInteger(row.product_id) END AS pid

  // Product events: ensure Product exists (fallback if products weren't imported yet)
  CALL {
    WITH row, pid
    WITH row, pid WHERE pid IS NOT NULL
    MERGE (p:Product {id: pid})
    ON CREATE SET p.product_type = coalesce(row.product_type, 'LAPTOP')
    RETURN p
    UNION
    WITH row, pid
    WITH row, pid WHERE pid IS NULL
    RETURN null AS p
  }
  WITH row, ts, u, p

  // 1) SEARCHED -> Query
  FOREACH (_ IN CASE WHEN row.event_type = 'search' AND row.query_text IS NOT NULL AND
      row.query_text <> '' THEN [1] ELSE [] END |
    MERGE (q:Query {text: row.query_text})
    MERGE (u)-[r:SEARCHED]->(q)
    ON CREATE SET r.cnt = 1, r.last_ts = ts, r.w = 1
    ON MATCH  SET r.cnt = coalesce(r.cnt, 0) + 1,
                  r.last_ts = ts,
                  r.w = coalesce(r.w, 0) + 1
  )

  // 2) FILTERED -> Query
  FOREACH (_ IN CASE WHEN row.event_type = 'filter' AND row.query_text IS NOT NULL AND
      row.query_text <> '' THEN [1] ELSE [] END |
    MERGE (q:Query {text: row.query_text})
    MERGE (u)-[r:FILTERED]->(q)
    ON CREATE SET r.cnt = 1, r.last_ts = ts, r.w = 1
    ON MATCH  SET r.cnt = coalesce(r.cnt, 0) + 1,
                  r.last_ts = ts,
                  r.w = coalesce(r.w, 0) + 1
  )
```

```
// 3) SORTED -> Query
FOREACH (_ IN CASE WHEN row.event_type = 'sort' AND row.query_text IS NOT NULL AND
    row.query_text <> '' THEN [1] ELSE [] END |
  MERGE (q:Query {text: row.query_text})
  MERGE (u)-[r:SORTED]->(q)
  ON CREATE SET r.cnt = 1, r.last_ts = ts, r.w = 1
  ON MATCH  SET r.cnt = coalesce(r.cnt, 0) + 1,
                r.last_ts = ts,
                r.w = coalesce(r.w, 0) + 1
)

// 4) PAGINATED -> Page
FOREACH (_ IN CASE WHEN row.event_type = 'paginate' AND row.page IS NOT NULL AND row.page
    <> '' THEN [1] ELSE [] END |
  MERGE (pg:Page {name: row.page})
  MERGE (u)-[r:PAGINATED]->(pg)
  ON CREATE SET r.cnt = 1, r.last_ts = ts, r.w = 1
  ON MATCH  SET r.cnt = coalesce(r.cnt, 0) + 1,
                r.last_ts = ts,
                r.w = coalesce(r.w, 0) + 1
)

// 4a) WISHLISTED -> Product (w += 2)
FOREACH (_ IN CASE WHEN row.event_type = 'add_to_wishlist' AND p IS NOT NULL THEN [1] ELSE
    [] END |
  MERGE (u)-[r:WISHLISTED]->(p)
  ON CREATE SET r.cnt = 1, r.last_ts = ts, r.w = 2
  ON MATCH  SET r.cnt = coalesce(r.cnt, 0) + 1,
                r.last_ts = ts,
                r.w = coalesce(r.w, 0) + 2
)

// 5) VIEWED -> Product (w += 1)
FOREACH (_ IN CASE WHEN row.event_type = 'view' AND p IS NOT NULL THEN [1] ELSE [] END |
  MERGE (u)-[r:VIEWED]->(p)
  ON CREATE SET r.cnt = 1, r.last_ts = ts, r.w = 1
  ON MATCH  SET r.cnt = coalesce(r.cnt, 0) + 1,
                r.last_ts = ts,
                r.w = coalesce(r.w, 0) + 1
)

// 6) CARTED -> Product (w += 3)
FOREACH (_ IN CASE WHEN row.event_type = 'add_to_cart' AND p IS NOT NULL THEN [1] ELSE []
    END |
  MERGE (u)-[r:CARTED]->(p)
  ON CREATE SET r.cnt = 1, r.last_ts = ts, r.w = 3
  ON MATCH  SET r.cnt = coalesce(r.cnt, 0) + 1,
                r.last_ts = ts,
                r.w = coalesce(r.w, 0) + 3
)

// 7) REMOVED_FROM_CART -> Product
FOREACH (_ IN CASE WHEN row.event_type = 'remove_from_cart' AND p IS NOT NULL THEN [1] ELSE
    [] END |
  MERGE (u)-[r:REMOVED_FROM_CART]->(p)
  ON CREATE SET r.cnt = 1, r.last_ts = ts, r.w = 1
  ON MATCH  SET r.cnt = coalesce(r.cnt, 0) + 1,
                r.last_ts = ts,
                r.w = coalesce(r.w, 0) + 1
)

// 8) STARTED_CHECKOUT -> Checkout
FOREACH (_ IN CASE WHEN row.event_type = 'checkout_start' THEN [1] ELSE [] END |
  MERGE (c:Checkout {id: row.event_id})
  SET c.ts = ts,
      c.page = row.page
  MERGE (u)-[:STARTED_CHECKOUT {ts: ts}]->(c)
)

// 9) PURCHASED -> Product (w += 10)
FOREACH (_ IN CASE WHEN row.event_type = 'purchase' AND p IS NOT NULL THEN [1] ELSE [] END |
  MERGE (u)-[r:PURCHASED]->(p)
  ON CREATE SET r.cnt = 1, r.last_ts = ts, r.w = 10
```

```
    ON MATCH  SET r.cnt = coalesce(r.cnt, 0) + 1,
                  r.last_ts = ts,
                  r.w = coalesce(r.w, 0) + 10
  )

  // 10) CHATTED -> ChatMessage
  FOREACH (_ IN CASE WHEN row.event_type = 'chat' AND row.query_text IS NOT NULL AND
      row.query_text <> '' THEN [1] ELSE [] END |
    MERGE (m:ChatMessage {id: row.event_id})
    SET m.text = row.query_text,
        m.page = row.page,
        m.product_type = row.product_type,
        m.ts = ts
    MERGE (u)-[:CHATTED {ts: ts}]->(m)
  )
} IN TRANSACTIONS OF 1000 ROWS
RETURN count(*) AS imported_rows;
```

4.4.2 Truy vấn context theo user_id và product_type

Khi chatbot nhận request, service gọi Neo4j để lấy top sản phẩm user tương tác mạnh nhất:

Listing 4.2: Cypher query lấy top products theo hành vi user (trích từ `chat/services.py`)

```
MATCH (u:User {id:
   $user_id})-[r:VIEWED|CARTED|WISHLISTED|PURCHASED]->(p:Product)
WHERE $product_type IS NULL OR p.product_type = $product_type
WITH p, sum(coalesce(r.w, 1)) AS score, max(coalesce(r.last_ts,
   datetime('1970-01-01T00:00:00Z'))) AS last_ts
RETURN p.id AS product_id,
       coalesce(p.name, 'Product #' + toString(p.id)) AS name,
       p.base_price AS base_price,
       coalesce(p.currency, 'VND') AS currency,
       score AS score
ORDER BY score DESC, last_ts DESC
LIMIT $limit
```

Trong đó `product_type` được lấy theo tab hiện tại từ frontend (mặc định `LAPTOP`).

4.5 Quy trình xử lý câu hỏi

Pipeline xử lý chính trong `ChatbotService.answer()` gồm:

1. Nhận `user_id`, `message`, `context`.
2. Xác định `product_type` theo `current_tab` (LAPTOP/CLOTHES).
3. Nếu user không phải guest: truy vấn Neo4j lấy `graph_context`.
4. Gọi `product_service` để lấy danh sách sản phẩm phù hợp (kết hợp search term + budget + intent).
5. Truy xuất top-k chunk từ FAISS (RAG sources).
6. Tạo prompt nạp: profile + context frontend + catalog products + graph context + RAG snippets.
7. Gọi Gemini (nếu có key); nếu lỗi/quota hoặc không có key thì fallback rule-based.

4.6 Mã nguồn cốt lõi của pipeline

4.6.1 Hàm `build_kb_index`: xây FAISS index cho knowledge base

Hàm này đọc các file markdown trong `knowledge_base/`, chunking, embedding (TF-IDF + SVD), rồi lưu FAISS index và metadata.

```
import faiss
import joblib
import numpy as np


def build_kb_index(force: bool = False):
    """Build FAISS index for markdown knowledge base (rút ọgn).

    Steps:
    1) Load + chunk KB markdown files
    2) Fit TF-IDF + TruncatedSVD embedder
    3) Embed all chunks -> dense vectors (L2-normalized)
    4) Build FAISS IndexFlatIP + persist index + metadata
    """

    docs = _load_kb_documents()  # [{id, document, metadata}, ...]
    if not docs:
        return {"indexed": 0, "reason": "knowledge base is empty"}

    if not force:
        existing = _load_faiss_store()
        if existing is not None and not _kb_is_stale():
            return {"indexed": 0, "reason": "faiss index already
               exists"}

    texts = [item["document"] for item in docs]
    embedder = _fit_embedder(texts)  # (vectorizer, svd)
    vectors = _embed_texts(embedder, texts)  # np.float32, shape (N,
       dim)

    dim = int(vectors.shape[1])
    index = faiss.IndexFlatIP(dim)
    index.add(np.asarray(vectors, dtype=np.float32))

    meta = [
        {"id": item["id"], "document": item["document"], "metadata":
           item["metadata"]}
        for item in docs
    ]

    _save_faiss_store(index, meta)
    joblib.dump(embedder, VECTORIZER_PATH)

    return {"indexed": len(docs), "dim": dim}
```

4.6.2 Hàm `fetch_graph_context`: truy vấn Neo4j lấy top products

Trong code thực tế, hàm tương ứng là `_fetch_graph_context`. Ở đây đổi tên để trình bày ngắn gọn.

```
from typing import Any


def fetch_graph_context(user_id: str, product_type: str | None,
   limit: int = 5) -> dict[str, Any]:
    """Fetch personalization context from Neo4j (rút ọgn).ấ

    Ly top ảsn ẩphm user ươtng tác ạmnh ấnht theo ổtng ọtrng ốs quan
       ệh.
    (Trong code ựthc ết: _fetch_graph_context)
    """

    driver = _get_neo4j_driver()
    if driver is None:
        return {}

    limit = max(1, min(int(limit), 20))
    product_type = product_type.strip().upper() if product_type else
       None

    cypher = """
MATCH (u:User {id:
   $user_id})-[r:VIEWED|CARTED|WISHLISTED|PURCHASED]->(p:Product)
WHERE $product_type IS NULL OR p.product_type = $product_type
WITH p, sum(coalesce(r.w, 1)) AS score, max(coalesce(r.last_ts,
   datetime('1970-01-01T00:00:00Z'))) AS last_ts
RETURN p.id AS product_id,
       coalesce(p.name, 'Product #' + toString(p.id)) AS name,
       p.base_price AS base_price,
       coalesce(p.currency, 'VND') AS currency,
       score AS score
ORDER BY score DESC, last_ts DESC
LIMIT $limit
""".strip()

    try:
        with driver.session() as session:
            top_products = [
                dict(record)
                for record in session.run(
                    cypher,
                    user_id=user_id,
                    product_type=product_type,
                    limit=limit,
                )
            ]
        return {"top_products": top_products, "brand_affinity": []}
```

```
    except Exception:
        return {}
```

4.6.3 Hàm `fetch_catalog_products`: gọi Product Service lấy catalog

Trong code thực tế, hàm tương ứng là:

`_fetch_catalog_products`

Hàm kết hợp intent/budget/search term, sau đó gọi REST tới `product_service`.

```
from typing import Any
import requests


def fetch_catalog_products(
    product_type: str,
    message: str,
    preferred_brands: list[str] | None = None,
    limit: int = 8,
) -> list[dict[str, Any]]:
    """Fetch products from Product Service (rút ọgn).

    (Trong code ựthc ết: _fetch_catalog_products)
    - Extract budget + detect intent
    - Build query term
    - Call product_service REST API and normalize output
    """

    if not PRODUCT_SERVICE_URL:
        return []

    base = PRODUCT_SERVICE_URL.rstrip("/") + "/api/v1/products/"

    budget = _extract_budget_vnd(message)
    intent = _detect_intent(message)
    q = (_intent_query(intent).strip() or
       _build_product_search_term(message)).strip()

    params: dict[str, Any] = {
        "product_type": product_type,
        "page_size": max(1, min(int(limit), 12)),
    }
    if q:
        params["q"] = q
    if budget is not None:
        params["min_price"] = int(budget * 0.70)
        params["max_price"] = int(budget * 1.30)

    try:
        r = requests.get(base, params=params, timeout=5)
```

```
        if r.status_code != 200:
            return []
        data = r.json() if r.content else {}
    except Exception:
        return []

    items = data.get("results", data) if isinstance(data, dict) else
       data
    if not isinstance(items, list):
        return []

    normalized: list[dict[str, Any]] = []
    for p in items[: max(1, min(int(limit), 12))]:
        brand = p.get("brand")
        normalized.append(
            {
                "id": p.get("id"),
                "name": p.get("name"),
                "brand": brand.get("name") if isinstance(brand,
                   dict) else brand,
                "base_price": p.get("base_price"),
                "currency": p.get("currency"),
                "in_stock": p.get("in_stock"),
                "thumbnail_url": p.get("thumbnail_url"),
            }
        )
    return normalized
```

4.6.4 Hàm `answer`: điều phối RAG + graph + catalog

Entrypoint chính của endpoint chat là:

```
ChatbotService.answer()
```

Hàm này lấy `graph_context`, `catalog_products`, `rag_sources` và sinh câu trả lời (Gemini hoặc fallback).

```
import time
from typing import Any


class ChatbotService:
    def __init__(self):
        self.behavior_profiles = _load_behavior_profiles()

    def answer(self, user_id: str, message: str, context: dict[str,
       Any] | None = None):
        """Main orchestration (rút ọgn).

        - Determine current_tab => product_type
        - Fetch Neo4j graph_context (if not guest)
```

```
        - Fetch catalog products from product_service
        - Retrieve KB chunks (FAISS)
        - Generate by Gemini or fallback
        """

        started = time.time()
        context = context or {}

        tab = str(context.get("current_tab") or
           "LAPTOP").strip().upper()
        product_type = "CLOTHES" if tab == "CLOTHES" else "LAPTOP"

        profile = self.behavior_profiles.get(user_id) or {
            "user_id": user_id,
            "segment": "new_user",
            "brand_preference": [],
        }

        graph_context: dict[str, Any] = {}
        if user_id and user_id != "guest_user":
            graph_context = fetch_graph_context(user_id=user_id,
               product_type=product_type, limit=5)

        catalog_products = fetch_catalog_products(
            product_type=product_type,
            message=message,
            preferred_brands=profile.get("brand_preference", []),
            limit=8,
        )

        chunks = _retrieve_chunks(message,
           user_segment=profile.get("segment", "new_user"))
        gemini_answer = _generate_with_gemini(message, profile,
           context, chunks, catalog_products, graph_context)
        answer_text = gemini_answer or _fallback_answer(message,
           profile, chunks, catalog_products)

        duration_ms = int((time.time() - started) * 1000)

        return {
            "answer": answer_text,
            "product_type": product_type,
            "recommended_products": catalog_products[:3],
            "rag_sources": [chunk.__dict__ for chunk in chunks],
            "graph_context": graph_context,
            "processing_time_ms": duration_ms,
            "fallback_mode": gemini_answer is None,
        }
```

4.7 Tích hợp context vào prompt và ràng buộc đề xuất

Prompt gửi LLM có các ràng buộc quan trọng:

- **Chỉ được đề xuất sản phẩm có trong catalog** do `product_service` trả về.
- Nếu hãng yêu thích không có đủ sản phẩm, phải nói rõ và gợi ý hãng khác nhưng vẫn nằm trong catalog.

4.8 Ví dụ minh hoạ (bằng chứng từ hệ thống)

Hình 4.3 là response JSON thực tế (lưu tại `assets/chat_response.json`) cho câu hỏi: *Mình cần laptop văn phòng tầm 20 triệu, nên chọn gì?*.

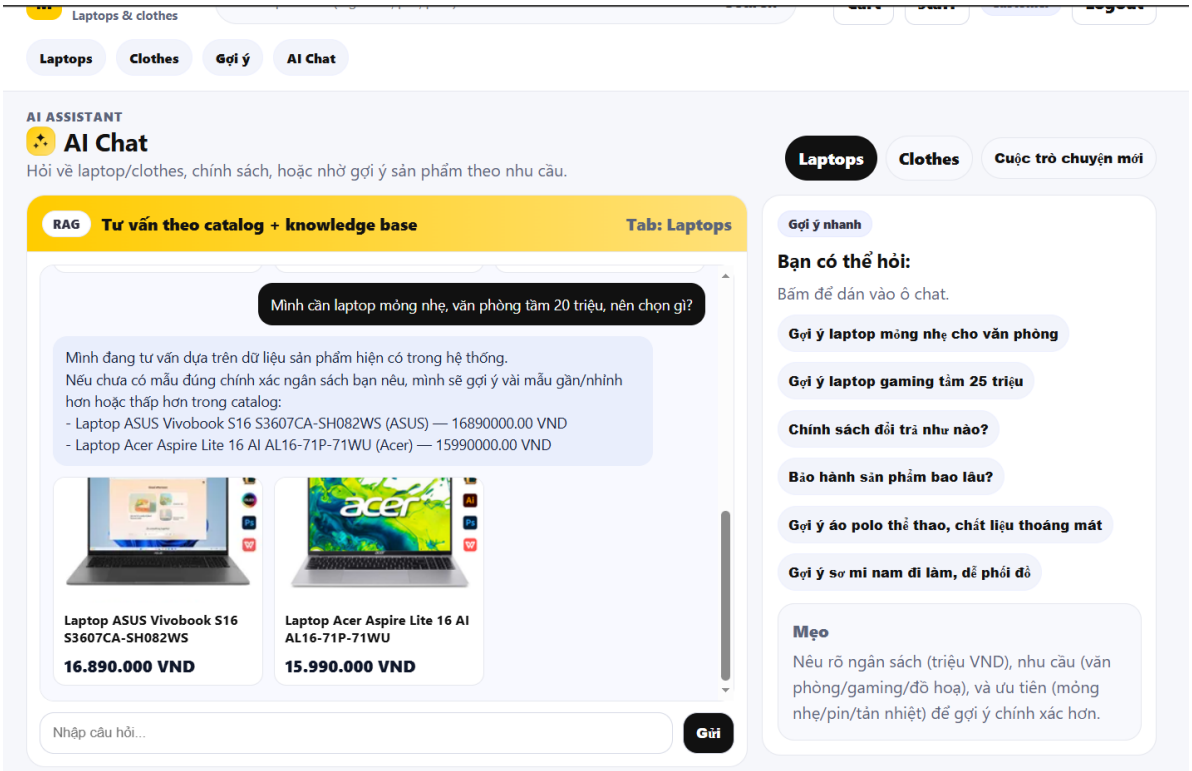

Figure 4.2: Giao diện chatbot trong API Gateway

Listing 4.3: Response JSON (rút gọn) minh hoạ câu trả lời của chatbot

```
(Trích file: ai_chat_service/assets/chat_response.json)
- answer: ...
- recommended_products: [ ... ]
- rag_sources: [ ... ]
- graph_context: { top_products: ... }
- fallback_mode: true
```

Trong response:

- `rag_sources` cho biết các tài liệu KB đã được truy xuất (ví dụ: *choose_by_usage*, *technical_faq*).

- `graph_context.top_products` cho biết các product user tương tác mạnh (dựa trên trọng số quan hệ).
- `recommended_products` chứa các sản phẩm thật lấy từ `product_service` để đảm bảo không bịa model.

4.9 Kết luận chương

GraphRAG trong AI Service (`ai_chat_service`) kết hợp tri thức (KB markdown) và hành vi (Neo4j) để tạo câu trả lời bám sát dữ liệu hệ thống. Thiết kế ưu tiên tính thực dụng/MVP: vector hoá nhẹ (TF-IDF+SVD) và cơ chế fallback giúp service vẫn hoạt động khi LLM không sẵn sàng.

Chapter 5

Tích hợp AI Service vào hệ thống E-commerce

5.1 Vai trò trong API Gateway

Trong kiến trúc microservices, `api_gateway` chịu trách nhiệm render HTML templates và điều phối gọi REST đến các service phía sau. AI Service được tích hợp theo hướng:

- Frontend gửi message đến gateway.
- Gateway gọi endpoint của `ai_chat_service` để lấy kết quả AI.
- Response trả về (tuỳ luồng) gồm: câu trả lời, danh sách sản phẩm gợi ý, nguồn RAG, và/hoặc graph context.

5.2 Tính năng gợi ý sản phẩm theo lịch sử tương tác (Recommendations)

Ngoài chatbot, hệ thống còn hỗ trợ gợi ý sản phẩm dựa trên lịch sử tương tác user (`view/cart/wishlist/purchase`). Trong API Gateway, trang `/recommendations/` triển khai theo hướng:

1. Gateway gửi một message hệ thống (ví dụ: *"recommend products based on user interaction history"*) đến endpoint chat của `ai_chat_service`.
2. AI service trả về `graph_context.top_products` (top sản phẩm theo trọng số quan hệ trong Neo4j) và `recommended_products` (danh sách sản phẩm từ `product_service`).
3. Gateway ưu tiên render các sản phẩm gợi ý, sau đó backfill từ `product_service` để đảm bảo đủ số lượng hiển thị.

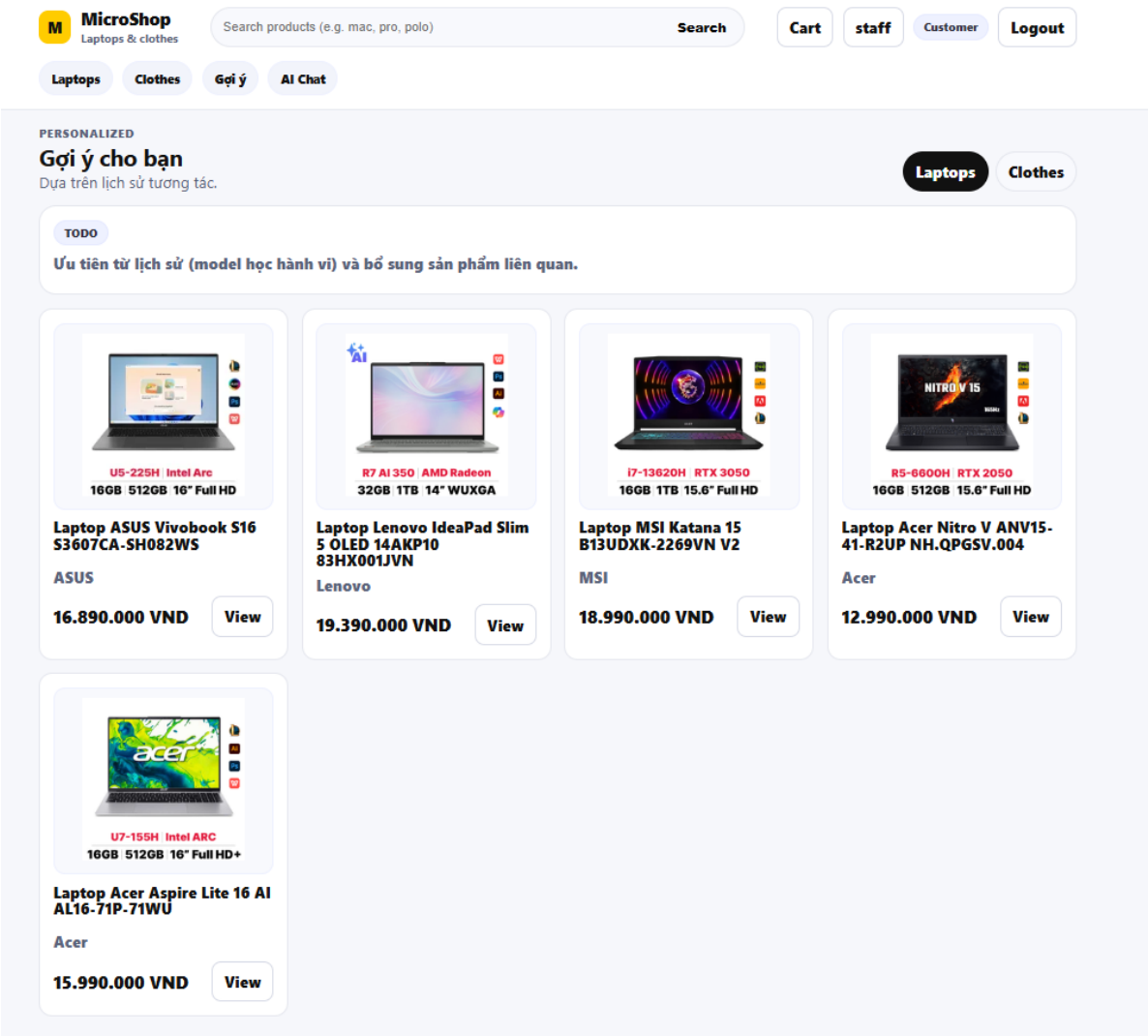

Figure 5.1: Minh hoạ gợi ý sản phẩm dựa trên lịch sử tương tác

5.3 Tính năng chatbot AI (RAG)

5.3.1 Giao diện và trải nghiệm người dùng

Giao diện chat được thiết kế theo hướng thân thiện, tối ưu cho e-commerce:

- Cửa sổ chat cố định, dễ truy cập khi browsing.
- Hiển thị các sản phẩm đề xuất trực tiếp trong câu trả lời (tên + giá + hình).
- Gợi ý hành động (ví dụ: mở category, apply filter, search) để dẫn dắt user theo funnel.

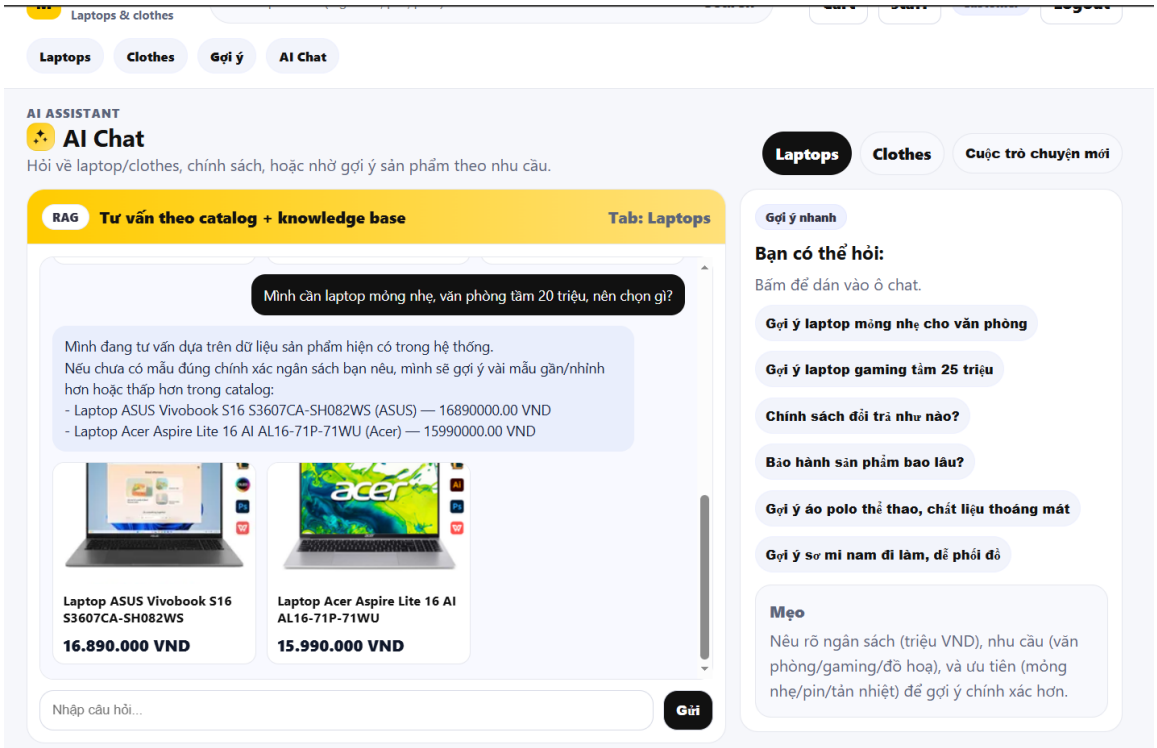

Figure 5.2: Minh hoạ giao diện chatbot trong hệ thống

5.3.2 Các tình huống hỗ trợ mua sắm

Chatbot hỗ trợ nhiều tình huống thực tế:

- **Tư vấn theo ngân sách**: trích xuất ngân sách từ câu hỏi (ví dụ: `20 triệu`) và lọc sản phẩm phù hợp.
- **Tư vấn theo nhu cầu**: phát hiện intent như mỏng nhẹ / đồ hoạ-kỹ thuật và ưu tiên sản phẩm tương ứng.
- **Hỏi đáp chính sách**: dùng RAG để trả lời theo knowledge base (đổi trả, bảo hành,...).
- **Cá nhân hoá**: dùng graph context để ưu tiên nhóm sản phẩm user đã quan tâm.

5.4 Đánh giá tích hợp

Việc tích hợp AI Service vào hệ thống giúp:

- Tăng khả năng **tự phục vụ** (self-service) của người dùng khi mua sắm.
- Hạn chế câu trả lời sai lệch bằng ràng buộc **chỉ đề xuất sản phẩm trong catalog**.
- Mở đường cho các mở rộng: dùng embedding model mạnh hơn, thêm reranking, và tối ưu truy vấn Neo4j theo product graph.

5.5 Kết luận

AI Service đóng vai trò như một **trợ lý mua sắm** và **bộ máy gợi ý** trong kiến trúc microservices. Service kết hợp RAG (FAISS + knowledge base) và đồ thị hành vi (Neo4j) để nâng chất lượng tư vấn, đồng thời đảm bảo tính nhất quán với dữ liệu sản phẩm thật trong hệ thống.